

# PRUEBA TÉCNICA - ANDROID (JETPACK COMPOSE)

## Perfil objetivo

Android Developer Semi-Senior / Senior. Jetpack Compose es OBLIGATORIO.

## Contexto

Construye una aplicación Android nativa con Jetpack Compose que consuma una API pública y permita al usuario explorar y guardar contenido. La UI debe estar construida íntegramente con Compose; no se aceptan layouts XML.

## Requisitos funcionales

14. Pantalla 1: lista de elementos consumidos desde una API pública (sugerencias: TMDb, PokeAPI, RickAndMortyAPI, OpenLibrary, o la que prefieras).
15. Pantalla 2: detalle del elemento al hacer tap.
16. Paginación o scroll infinito en la lista.
17. Búsqueda local o remota sobre los elementos.
18. Marcar elementos como favoritos. Los favoritos deben persistir entre sesiones.
19. Pantalla 3 (o tab/sección): lista de favoritos.
20. Manejo apropiado de estados de carga, error y vacío.

## Requisitos técnicos (obligatorios)

- Kotlin (no Java).
- Jetpack Compose para TODA la UI. No se aceptan layouts XML ni interop con Views salvo justificación muy fuerte.
- Material 3 (androidx.compose.material3).
- Navigation Compose para la navegación entre pantallas.
- Arquitectura MVVM o Clean Architecture con separación clara de capas.
- Inyección de dependencias: Hilt o Koin.
- Consumo de API con Retrofit + corrutinas o Flow.
- Persistencia con Room para los favoritos.
- Manejo de estado con StateFlow. Usar collectAsStateWithLifecycle() en Compose, no collectAsState() directo.
- Uso correcto de remember, rememberSaveable y derivedStateOf donde corresponda.
- ViewModels que sobrevivan a configuration changes.
- Al menos 3-5 tests significativos (unitarios de ViewModel/repositorio y/o tests de Compose con createComposeRule).
- minSdk razonable (24 o superior es buena idea).

## Bonus (opcional)

- Paging 3 integrado con Compose.
- Tests instrumentados con Compose Testing.

- Animaciones nativas de Compose (AnimatedVisibility, animateContentSize, etc.).
- Modo oscuro con MaterialTheme y dynamic color (Material You).
- Coil para carga de imágenes.
- CI básico (GitHub Actions).
- Compose Multiplatform si te animas (no obligatorio).

## Lo que NO necesitas hacer

- Backend propio.
- Autenticación con cuentas reales.
- Publicación en Play Store.
- Compatibilidad con versiones antiguas de Android (minSdk 24+ está bien).

## Banderas rojas que evaluaremos

- Uso de findViewById, View binding o cualquier rastro de XML para UI.
- LiveData en lugar de StateFlow sin justificación.
- collectAsState() en lugar de collectAsStateWithLifecycle().
- Lógica de negocio dentro de @Composable functions.
- Recomposiciones evidentemente innecesarias (no usar remember donde toca).
- ViewModels que filtran tipos de Compose o Context de Android.

## Instrucciones generales

- Plazo de entrega: 3 días corridos a partir de la recepción de este documento.
- Entrega: enviar repositorio público de GitHub (o privado dándonos acceso) con el código y un README claro.
- El README debe incluir: instrucciones de instalación y ejecución, decisiones técnicas tomadas, trade-offs, lista de lo que no alcanzó a hacer y por qué.
- Se valora más calidad que cantidad. Si no alcanzas a hacer todo, prioriza terminar bien lo que sí entregues.
- Puedes usar las librerías que consideres apropiadas, justificando la decisión en el README.
- No copies código de internet sin entenderlo. En la entrevista te pediremos explicar tus decisiones.

## Criterios de evaluación

- Funcionalidad: que cumpla los requisitos y funcione.
- Calidad de código: legibilidad, nombres, estructura.
- Arquitectura: patrones, separación de responsabilidades, escalabilidad.
- Testing: tests unitarios o de integración sobre la lógica crítica.
- Manejo de errores: validaciones, casos borde.
- Documentación: README y decisiones técnicas.